AWS Neptune Graph Database Architecture

Overview

Graph database architecture using AWS Neptune for relationship-heavy data, social networks, recommendation engines, fraud detection, and knowledge graphs.

What is a Graph Database?

Graph databases store data as **nodes** (entities) and **edges** (relationships), making them ideal for:

- Social networks (friends, followers)
- Recommendation engines (user-product relationships)
- Fraud detection (transaction patterns)
- Knowledge graphs (entity relationships)
- Network topology (infrastructure dependencies)

Graph vs. Relational

Relational (JOIN-heavy):

```
-- Find friends of friends (2 hops)
SELECT u3.name
FROM users u1
JOIN friendships f1 ON u1.id = f1.user_id
JOIN users u2 ON f1.friend_id = u2.id
JOIN friendships f2 ON u2.id = f2.user_id
JOIN users u3 ON f2.friend_id = u3.id
WHERE u1.id = 123;
-- Performance degrades exponentially with depth
```

Graph (Traversal):

```
// Gremlin/Cypher - constant time per hop
MATCH (u:User {id: 123})-[:FRIEND*2]-(friend_of_friend)
RETURN friend_of_friend.name;
```

AWS Neptune Overview

Supported Query Languages

1. Gremlin (Apache TinkerPop)

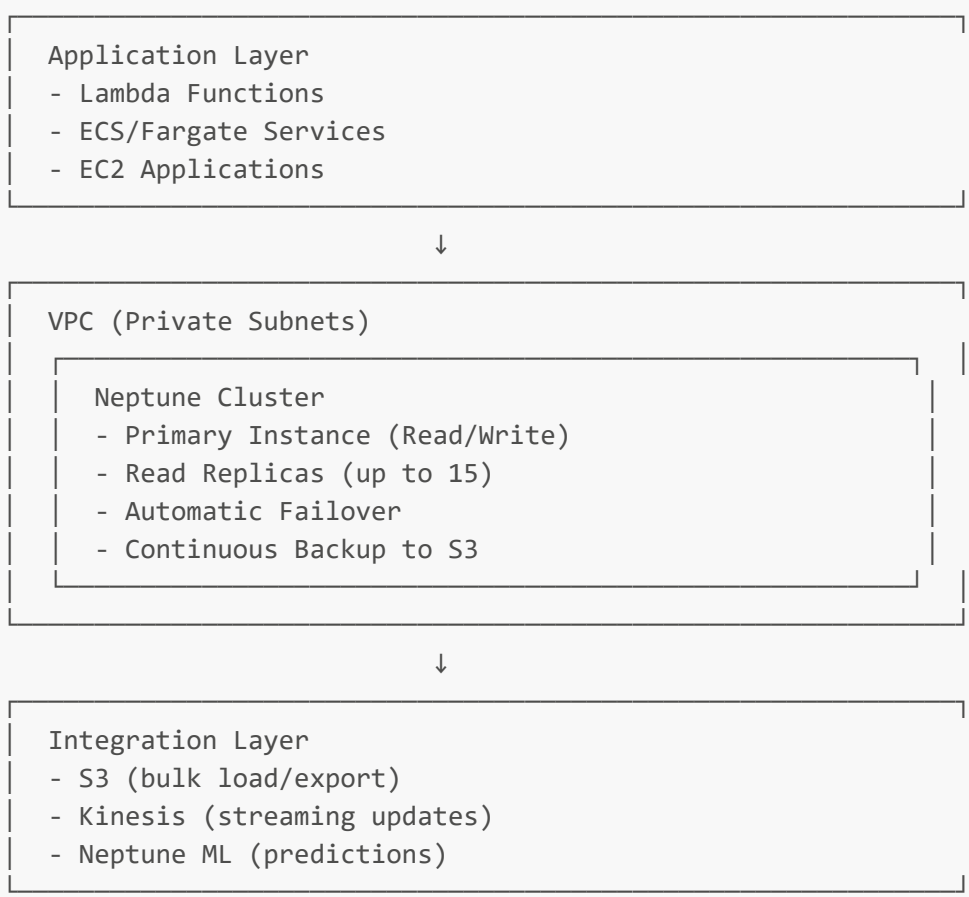
- Property graph model

- Traversal-based queries
- More flexible for complex patterns

2. SPARQL (RDF)

- Semantic web standard
- Triple store (subject-predicate-object)
- Best for ontologies and knowledge graphs

Architecture



Data Modeling

Property Graph Model (Gremlin)

Example: Social Network

```
from gremlin_python.structure.graph import Graph
from gremlin_python.driver.driver_remote_connection import DriverRemoteConnection
from gremlin_python.process.anonymous_traversal import traversal

# Connect to Neptune
graph = Graph()
```

```
connection = DriverRemoteConnection(
    'wss://your-neptune-endpoint:8182/gremlin',
    'g'
)
g = traversal().withRemote(connection)

# Create vertices (nodes)
user1 = g.addV('User') \
    .property('id', '1') \
    .property('name', 'Alice') \
    .property('age', 30) \
    .property('city', 'San Francisco') \
    .next()

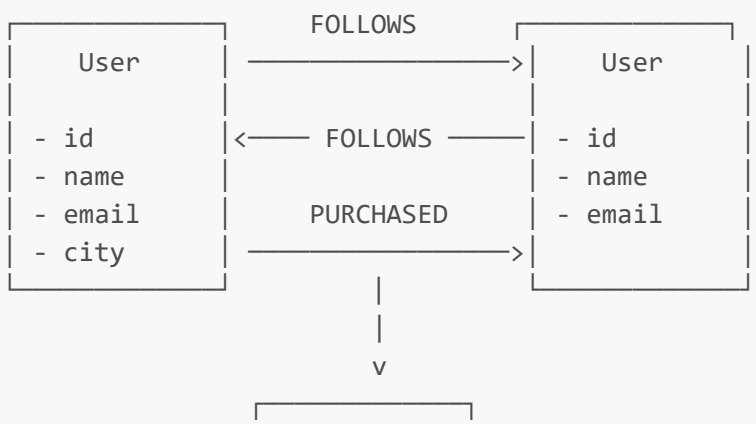
user2 = g.addV('User') \
    .property('id', '2') \
    .property('name', 'Bob') \
    .property('age', 25) \
    .property('city', 'New York') \
    .next()

product1 = g.addV('Product') \
    .property('id', 'p1') \
    .property('name', 'Laptop') \
    .property('category', 'Electronics') \
    .property('price', 1200) \
    .next()

# Create edges (relationships)
g.V(user1).addE('FOLLOWS').to(user2) \
    .property('since', '2024-01-01') \
    .next()

g.V(user1).addE('PURCHASED').to(product1) \
    .property('date', '2024-03-15') \
    .property('quantity', 1) \
    .next()
```

Graph Schema Example



```
                    ┌──────────────┐
                    │   Product    │
                    │              │
                    │ - id         │
                    │ - name       │
                    │ - price      │
                    │ - category   │
                    └──────────────┘
                           ^
                           |
                      BELONGS_TO
                           |
                    ┌──────────────┐
                    │  Category    │
                    │ - id         │
                    │ - name       │
                    └──────────────┘
```

Common Query Patterns

1. Find Friends (1 hop)

```
# Gremlin
friends = g.V().has('User', 'id', '1') \
    .out('FOLLOWS') \
    .values('name') \
    .toList()
```

2. Friends of Friends (2 hops)

```
# Exclude self and direct friends
fof = g.V().has('User', 'id', '1') \
    .out('FOLLOWS').aggregate('friends') \
    .out('FOLLOWS') \
    .where(without('friends')) \
    .where(neq('1')) \
    .dedup() \
    .values('name') \
    .toList()
```

3. Shortest Path

```
# Find shortest path between two users
path = g.V().has('User', 'id', '1') \
    .repeat(out('FOLLOWS').simplePath()) \
    .until(has('id', '10')) \
```

```
    .path() \
    .limit(1) \
    .next()
```

4. Recommendation Engine

```
# Recommend products based on friends' purchases
recommendations = g.V().has('User', 'id', '1') \
    .out('FOLLOWS') \
    .out('PURCHASED') \
    .where(without(['self_purchased'])) \
    .groupCount() \
    .order(local).by(values, desc) \
    .limit(10) \
    .toList()

# More sophisticated: collaborative filtering
similar_users = g.V().has('User', 'id', '1') \
    .out('PURCHASED').aggregate('my_products') \
    .in_('PURCHASED') \
    .where(neq('1')) \
    .where(out('PURCHASED').where(within('my_products')).count().is_(gte(3))) \
    .dedup() \
    .toList()

recommendations = g.V(similar_users) \
    .out('PURCHASED') \
    .where(without('my_products')) \
    .groupCount().by('name') \
    .order(local).by(values, desc) \
    .limit(10) \
    .toList()
```

5. Fraud Detection Pattern

```
# Find suspicious circular money transfers
suspicious = g.V().has('Account', 'id', 'A123') \
    .repeat(
        outE('TRANSFERRED').has('amount', gt(1000))
        .inV().simplePath()
    ) \
    .until(has('id', 'A123')) \
    .path() \
    .toList()

# Find accounts with unusual connection patterns
anomalies = g.V().hasLabel('Account') \
    .where(
        out('TRANSFERRED').count().is_(gt(100)).and_()
```

```
            .out('TRANSFERRED').values('amount').mean().is_(gt(5000))
        ) \
        .valueMap(True) \
        .toList()
```

6. Influence Score (PageRank-like)

```
# Users with most followers
influencers = g.V().hasLabel('User') \
    .project('user', 'follower_count') \
    .by('name') \
    .by(in_('FOLLOWS').count()) \
    .order().by('follower_count', desc) \
    .limit(10) \
    .toList()
```

Data Loading

1. Bulk Load from S3

```
import boto3
import json

# Prepare data in Neptune CSV format
# vertices.csv
"""
~id,~label,name:String,age:Int,city:String
1,User,Alice,30,San Francisco
2,User,Bob,25,New York
p1,Product,Laptop,1200,Electronics
"""

# edges.csv
"""
~id,~from,~to,~label,since:String
e1,1,2,FOLLOWS,2024-01-01
e2,1,p1,PURCHASED,2024-03-15
"""

# Upload to S3
s3 = boto3.client('s3')
s3.upload_file('vertices.csv', 'my-bucket', 'neptune-data/vertices.csv')
s3.upload_file('edges.csv', 'my-bucket', 'neptune-data/edges.csv')

# Trigger Neptune Loader
neptune = boto3.client('neptunedata')

response = neptune.start_loader_job(
```

```
    source='s3://my-bucket/neptune-data/',
    format='csv',
    s3BucketRegion='us-east-1',
    iamRoleArn='arn:aws:iam::123456789:role/NeptuneLoadRole',
    mode='NEW'  # or 'RESUME', 'AUTO'
)

load_id = response['payload']['loadId']

# Check status
status = neptune.get_loader_job_status(loadId=load_id)
print(status['payload']['overallStatus']['status'])  # LOAD_COMPLETED
```

2. Streaming Updates with Kinesis

```
# Lambda function triggered by Kinesis
def lambda_handler(event, context):
    g = get_neptune_connection()

    for record in event['Records']:
        data = json.loads(base64.b64decode(record['kinesis']['data']))

        if data['type'] == 'user_action':
            # Add interaction edge
            g.V().has('User', 'id', data['user_id']) \
                .addE('INTERACTED') \
                .to(g.V().has('Product', 'id', data['product_id'])) \
                .property('action', data['action']) \
                .property('timestamp', data['timestamp']) \
                .next()
```

3. Real-time Updates from Application

```
class NeptuneGraphService:

    def __init__(self, endpoint: str):
        self.g = self.connect(endpoint)

    def connect(self, endpoint: str):
        connection = DriverRemoteConnection(
            f'wss://{endpoint}:8182/gremlin',
            'g'
        )
        return traversal().withRemote(connection)

    def add_user(self, user_id: str, name: str, **properties):
        """Add a new user vertex."""
        vertex = self.g.addV('User') \
            .property('id', user_id) \
```

```
            .property('name', name)

        for key, value in properties.items():
            vertex = vertex.property(key, value)

        return vertex.next()

    def add_friendship(self, user_id1: str, user_id2: str, bidirectional: bool =
True):
        """Create friendship relationship."""
        # Add edge from user1 to user2
        self.g.V().has('User', 'id', user_id1) \
            .addE('FOLLOWS').to(
                self.g.V().has('User', 'id', user_id2)
            ) \
            .property('since', datetime.now().isoformat()) \
            .next()

        if bidirectional:
            # Add edge from user2 to user1
            self.g.V().has('User', 'id', user_id2) \
                .addE('FOLLOWS').to(
                    self.g.V().has('User', 'id', user_id1)
                ) \
                .property('since', datetime.now().isoformat()) \
                .next()

    def get_user_network(self, user_id: str, depth: int = 2):
        """Get user's network up to specified depth."""
        return self.g.V().has('User', 'id', user_id) \
            .repeat(
                out('FOLLOWS').simplePath()
            ).times(depth) \
            .dedup() \
            .valueMap(True) \
            .toList()

    def recommend_connections(self, user_id: str, limit: int = 10):
        """Recommend new connections (friends of friends)."""
        return self.g.V().has('User', 'id', user_id) \
            .out('FOLLOWS').aggregate('friends') \
            .out('FOLLOWS') \
            .where(without('friends')) \
            .where(neq(user_id)) \
            .dedup() \
            .project('user', 'mutual_friends') \
            .by(valueMap(True)) \
            .by(in_('FOLLOWS').where(within('friends')).count()) \
            .order().by('mutual_friends', desc) \
            .limit(limit) \
            .toList()
```

Integration with ML/LLM

1. Neptune ML for Link Prediction

```
import boto3

# Export graph for ML
neptune_ml = boto3.client('neptunedata')

# Create export job
export_response = neptune_ml.create_ml_data_processing_job(
    id='link-prediction-export',
    processingInstanceType='ml.r5.xlarge',
    s3OutputEncryptionKMSKey='your-kms-key',
    processingInstanceVolumeSizeInGB=50,
    outputDataS3Location='s3://my-bucket/neptune-ml/output'
)

# Train model with SageMaker
sagemaker = boto3.client('sagemaker')

training_job = sagemaker.create_training_job(
    TrainingJobName='neptune-link-prediction',
    AlgorithmSpecification={
        'TrainingImage': 'neptune-ml-image',
        'TrainingInputMode': 'File'
    },
    RoleArn='arn:aws:iam::123456789:role/SageMakerRole',
    InputDataConfig=[{
        'ChannelName': 'training',
        'DataSource': {
            'S3DataSource': {
                'S3Uri': 's3://my-bucket/neptune-ml/output',
                'S3DataType': 'S3Prefix'
            }
        }
    }],
    OutputDataConfig={
        'S3OutputPath': 's3://my-bucket/neptune-ml/model'
    },
    ResourceConfig={
        'InstanceType': 'ml.p3.2xlarge',
        'InstanceCount': 1,
        'VolumeSizeInGB': 50
    }
)

# Create inference endpoint
inference_response = neptune_ml.create_ml_endpoint(
    id='link-prediction-endpoint',
    mlModelTrainingJobId='link-prediction-model'
)
```

```
# Use predictions
prediction = g.V().has('User', 'id', '1') \
    .out('FOLLOWS') \
    .out('FOLLOWS') \
    .where(without(['direct_friends'])) \
    .with_('Neptune#ml.endpoint', 'link-prediction-endpoint') \
    .with_('Neptune#ml.score', 'prediction') \
    .order().by('prediction', desc) \
    .limit(10) \
    .toList()
```

2. LLM-Enhanced Graph Queries

```
from langchain_community.llms import Bedrock
from langchain.chains import GraphCypherQAChain

class NeptuneQueryAssistant:

    def __init__(self, neptune_endpoint: str):
        self.g = connect_neptune(neptune_endpoint)
        self.llm = Bedrock(model_id="anthropic.claude-3-sonnet-20240229-v1:0")

    def natural_language_query(self, question: str) -> dict:
        """Convert natural language to Gremlin query."""

        # Get graph schema
        schema = self.get_schema()

        prompt = f"""Given this graph schema:

        {schema}

        Convert this question to a Gremlin query: {question}

        Return only the Gremlin query code.
        """

        gremlin_query = self.llm.invoke(prompt)

        # Execute query
        try:
            result = eval(f"self.g.{gremlin_query}.toList()")

            # Generate explanation
            explain_prompt = f"""
            Query: {question}
            Gremlin: {gremlin_query}
            Results: {result}

            Provide a natural language summary of these results.
```

```
            """

            explanation = self.llm.invoke(explain_prompt)

            return {
                "question": question,
                "gremlin": gremlin_query,
                "results": result,
                "explanation": explanation
            }
        except Exception as e:
            return {"error": str(e)}

    def get_schema(self) -> str:
        """Get graph schema for LLM context."""
        vertex_labels = self.g.V().label().dedup().toList()
        edge_labels = self.g.E().label().dedup().toList()

        schema = "Vertex Labels:\n"
        for label in vertex_labels:
            properties =
self.g.V().hasLabel(label).properties().key().dedup().toList()
            schema += f"  {label}: {', '.join(properties)}\n"

        schema += "\nEdge Labels:\n"
        for label in edge_labels:
            properties =
self.g.E().hasLabel(label).properties().key().dedup().toList()
            schema += f"  {label}: {', '.join(properties)}\n"

        return schema

# Usage
assistant = NeptuneQueryAssistant('your-neptune-endpoint')
result = assistant.natural_language_query(
    "Who are the most influential users in the network?"
)
```

3. Knowledge Graph with Embeddings

```
from langchain_community.embeddings import BedrockEmbeddings
import numpy as np

class KnowledgeGraphService:

    def __init__(self, neptune_endpoint: str):
        self.g = connect_neptune(neptune_endpoint)
        self.embeddings = BedrockEmbeddings(
            model_id="amazon.titan-embed-text-v1"
        )
```

```
    def add_entity_with_embedding(self, entity_type: str, name: str, description:
str):
        """Add entity with text embedding."""

        # Generate embedding
        embedding = self.embeddings.embed_query(description)

        # Store in Neptune
        self.g.addV(entity_type) \
            .property('name', name) \
            .property('description', description) \
            .property('embedding', embedding) \
            .next()

    def find_similar_entities(self, query: str, entity_type: str = None, top_k:
int = 5):
        """Find similar entities using embedding similarity."""

        # Generate query embedding
        query_embedding = self.embeddings.embed_query(query)

        # Get all entities
        query = self.g.V()
        if entity_type:
            query = query.hasLabel(entity_type)

        entities = query.valueMap(True).toList()

        # Calculate cosine similarity
        similarities = []
        for entity in entities:
            entity_embedding = entity['embedding']
            similarity = np.dot(query_embedding, entity_embedding) / \
                (np.linalg.norm(query_embedding) *
np.linalg.norm(entity_embedding))
            similarities.append({
                'entity': entity,
                'similarity': similarity
            })

        # Sort and return top-k
        similarities.sort(key=lambda x: x['similarity'], reverse=True)
        return similarities[:top_k]
```

Performance Optimization

1. Indexing

```
# Create composite index for faster lookups
# This is done at cluster creation time via parameter groups
```

```
# Use has() for indexed properties
optimized = g.V().has('User', 'email', 'alice@example.com')  # Fast

# Avoid filter() when possible
slow = g.V().hasLabel('User').filter_(lambda: "email == 'alice@example.com'")  #
Slow
```

2. Query Patterns

```
# Good: Specific starting point
good = g.V().has('User', 'id', '1').out('FOLLOWS').out('PURCHASED')

# Bad: Scanning all vertices
bad = g.V().hasLabel('User').out('FOLLOWS').out('PURCHASED')

# Use limits to prevent runaway queries
g.V().repeat(out()).times(5).limit(1000)

# Use .toList() instead of iterating
results = g.V().has('User', 'id', '1').out('FOLLOWS').toList()  # Single round
trip
```

3. Caching with ElastiCache

```
import redis
import pickle

class CachedNeptuneService:

    def __init__(self, neptune_endpoint: str, redis_endpoint: str):
        self.g = connect_neptune(neptune_endpoint)
        self.cache = redis.Redis(host=redis_endpoint, port=6379)

    def get_user_network(self, user_id: str, depth: int = 2):
        """Get user network with caching."""

        cache_key = f"network:{user_id}:{depth}"

        # Try cache
        cached = self.cache.get(cache_key)
        if cached:
            return pickle.loads(cached)

        # Query Neptune
        result = self.g.V().has('User', 'id', user_id) \
            .repeat(out('FOLLOWS')).times(depth) \
            .dedup() \
            .valueMap(True) \
```

```
            .toList()

        # Cache for 1 hour
        self.cache.setex(cache_key, 3600, pickle.dumps(result))

        return result
```

Monitoring & Maintenance

CloudWatch Metrics

```
import boto3

cloudwatch = boto3.client('cloudwatch')

# Key metrics to monitor
metrics = [
    'MainRequestQueuePendingRequests',  # Queue depth
    'GremlinRequestsPerSec',  # Query rate
    'SparqlRequestsPerSec',
    'CPUUtilization',
    'VolumeBytesUsed',
    'GremlinWebSocketOpenConnections'
]

# Create dashboard
dashboard_body = {
    "widgets": [{
        "type": "metric",
        "properties": {
            "metrics": [[
                "AWS/Neptune",
                metric,
                {"stat": "Average"}
            ] for metric in metrics],
            "period": 300,
            "region": "us-east-1",
            "title": "Neptune Cluster Health"
        }
    }]
}

cloudwatch.put_dashboard(
    DashboardName='Neptune-Monitor',
    DashboardBody=json.dumps(dashboard_body)
)
```

Backup & Restore

```
neptune = boto3.client('neptune')

# Create snapshot
neptune.create_db_cluster_snapshot(
    DBClusterSnapshotIdentifier='neptune-backup-2024-01-01',
    DBClusterIdentifier='my-neptune-cluster'
)

# Restore from snapshot
neptune.restore_db_cluster_from_snapshot(
    DBClusterIdentifier='my-neptune-cluster-restored',
    SnapshotIdentifier='neptune-backup-2024-01-01',
    Engine='neptune'
)
```

Use Case Examples

1. Social Network

- **Vertices**: Users, Posts, Comments
- **Edges**: FOLLOWS, LIKES, COMMENTS_ON, SHARES
- **Queries**: Friend recommendations, content feed, influencer detection

2. Fraud Detection

- **Vertices**: Accounts, Transactions, Devices, IP Addresses
- **Edges**: TRANSFERRED, ACCESSED_FROM, SIMILAR_TO
- **Queries**: Circular transfers, anomaly detection, risk scoring

3. Recommendation Engine

- **Vertices**: Users, Products, Categories
- **Edges**: PURCHASED, VIEWED, RATED, BELONGS_TO
- **Queries**: Collaborative filtering, similar products, personalized recommendations

4. Knowledge Graph

- **Vertices**: Entities, Concepts, Documents
- **Edges**: RELATED_TO, MENTIONS, PART_OF
- **Queries**: Entity linking, semantic search, relationship discovery

5. Supply Chain

- **Vertices**: Suppliers, Products, Warehouses, Customers
- **Edges**: SUPPLIES, STORED_IN, SHIPS_TO, REQUIRES
- **Queries**: Shortest path delivery, dependency analysis, bottleneck detection

---

Cost Optimization

1. **Use Read Replicas** for read-heavy workloads
2. **Right-size instances** (start with db.r5.large)
3. **Enable auto-scaling** for replicas
4. **Use caching** (ElastiCache) for frequent queries
5. **Optimize queries** to reduce traversal depth
6. **Monitor and alert** on unusual patterns
7. **Schedule backups** during off-peak hours

Migration Strategy

From Relational to Graph

```
def migrate_relational_to_graph():
    """Migrate from RDS to Neptune."""

    # Read from RDS
    import psycopg2
    conn = psycopg2.connect("postgresql://...")

    # Users
    users = pd.read_sql("SELECT * FROM users", conn)
    for _, user in users.iterrows():
        g.addV('User') \
            .property('id', user['id']) \
            .property('name', user['name']) \
            .property('email', user['email']) \
            .next()

    # Friendships
    friendships = pd.read_sql("SELECT * FROM friendships", conn)
    for _, friendship in friendships.iterrows():
        g.V().has('User', 'id', friendship['user_id']) \
            .addE('FOLLOWS').to(
                g.V().has('User', 'id', friendship['friend_id'])
            ) \
            .next()
```

Best Practices

1. ✅ Use specific vertex lookups (has()) instead of scanning
2. ✅ Add limits to prevent runaway queries
3. ✅ Use read replicas for analytics
4. ✅ Implement connection pooling
5. ✅ Cache frequent query results
6. ✅ Monitor query performance

7. ✅ Use bulk loading for initial data
8. ✅ Implement retry logic with exponential backoff
9. ✅ Version your graph schema
10. ✅ Test queries in dev before production